Research Landscape

Real-time Earthquake and Aftershock Detection using streaming data

Preview :

The proposed project will develop an earthquake monitoring and aftershock detecting system that is based on USGS data, which is in real-time streaming. The continuous streams of earthquake events will be analyzed in order to find possible clusters of aftershocks. All the raw data will be stored, cleansed, and enhanced within the lakehouse, making it possible for users to analyze the information about earthquake magnitude, depth, distribution, and aftershocks.

Key Findings And Comparisons :

Paper 1 :

Objectives : Assess the feasibility of using the Field Stream Data Model in detecting earthquake events in real time through the abstraction of discrete sensor data streams into continuous spatio-temporal fields as well as integration with the open-source Data Stream Engine (DSE).

Data Source : SCIGN (Southern California Integrated GPS Network) and USGS Earthquake Event Feed

Architecture :

- Sensor streams are modeled as temporal fields using the Field Stream Data Model and represented as Field RDDs in Apache Spark.
- Real-time GPS data is processed through Kafka-connected Spark Streaming jobs, where displacement changes across X/Y/Z coordinates are aggregated over temporal windows and compared against predefined thresholds for event detection.
- When an event is detected, spatial correlation with nearby sensors is performed to estimate its geographic extent and temporal boundaries.
- USGS earthquake events are used for post-hoc correlation by retrieving nearby SCIGN sensor data around the reported earthquake origin time.

Technologies : Apache Spark , Apache Kafka , MongoDB , Node.js , Socket.IO , NGINX and GeoJSON

Real-Time :

- Real-time stream processing is implemented using Apache Spark's micro-batch (DStream) model.
- SCIGN sensor data is processed using 5-second tumbling windows, with support for overlapping sliding windows.
- The system targets near-real-time earthquake detection and visualization with a 1–2 second processing delay.

Key Results :

- Demonstrated real-time earthquake detection from GPS streams using field abstractions.
- Detected events correlated with USGS data and visualized in real time.
- Modular architecture supports independent component scaling.
- Field streams effectively implemented using RDDs + function libraries.

Solution (Paper 2) - USGS Earthquake Sequence Product :

The USGS Earthquake Sequence Product groups and displays earthquakes that are closely related in space and time. This helps scientists and the public understand how earthquakes are connected and how they unfold. By looking at these sequences, users can see patterns like where aftershocks occur, how strong they are, and how often they happen.

The USGS has an earthquake sequence product and an aftershock forecast product. The Sequence Product differs from the Aftershock Forecast product in that it shows what has happened or is happening in an earthquake sequence, while the Aftershock Forecast predicts what is likely to happen next

The USGS automatically classifies earthquakes as being part of a sequence by leveraging the same statistical models used by the USGS Aftershock Forecast Product.

GitHub Solution 1 :

Objectives : Detect earthquake/ground-motion events using distributed physical seismic stations and perform initial processing at the edge.

Data Source : Physical seismic sensors connected to GeoShake T1 stations. The station uses accelerometers to measure ground motion.

Architecture : IoT/Edge architecture: Accelerometers → ESP32-S3 → on-device signal processing/detection → Wi-Fi → MQTT/TLS → server/network.

Technologies : ESP32-S3, accelerometers, ESP-IDF, STA/LTA algorithm, Wi-Fi, MQTT over TLS, waveform processing and mobile/network components.

Real-Time : Real-time edge detection: the device continuously measures ground motion and performs detection locally.

Key Results : Detects seismic activity/P-wave triggers and provides seismic measurements/waveform information from physical stations.

Pros:

- Real-time seismic detection using physical sensors
- Edge-based processing enables low-latency earthquake detection
- Captures actual ground-motion and waveform data
- Distributed sensor architecture supports localized earthquake monitoring

Cons:

- Requires physical hardware and deployment of seismic stations
- Geographic coverage depends on sensor placement
- No spatiotemporal correlation for identifying potential aftershock clusters
- No lakehouse for large-scale historical earthquake and aftershock analysis

Similar GitHub Solution 2 :

Objectives : Stream live earthquake data from the USGS API into a database, replicate every change in real time via CDC, and visualize global seismic activity on dashboards.

Data Source : USGS FDSN Event API — polled every 60 seconds.

Architecture : USGS API → MySQL → Debezium (CDC) → Kafka → JDBC Sink → PostgreSQL → Grafana

Technologies (exact stack) : Python (ingestion script), MySQL (source DB, binlog enabled), Debezium (CDC), Apache Kafka + Kafka Connect (JDBC Sink), PostgreSQL (sink DB), Grafana (visualization), Kafka UI and Adminer (monitoring), Docker Compose (orchestration, single host).

Real-time : Polling-based, same as the source — checks USGS every 60 seconds; once inside MySQL, CDC propagation to Kafka/PostgreSQL is near-instant via binlog.

Key Results :

- Working end-to-end demo (`docker compose up` and it runs)
- Live world map, quakes-per-hour trend, top hotspot regions, live activity gauge
- Demonstrates CDC as a general reusable pattern (they cite IoT, inventory, fraud, healthcare as other use cases)

Pros :

- Fully reproducible, clean working demo — good engineering hygiene
- Efficient CDC pattern (binlog-based) for keeping databases in sync
- Simple, effective dashboard set
- Good generalizability story (explicitly ties the pattern to other domains)

Cons :

- No intelligence/correlation — doesn't ask "are these events related?"
- No lakehouse, so no real historical pattern analysis at scale
- Not cloud-native — single machine, no scaling under load
- CDC is plumbing, not processing — a simpler problem than stateful stream analytics
- Same 60-second polling ceiling, so "real-time" claim carries the same caveat

Identified Gap and Proposed Direction:

	Identified Gap	Proposed Solution
Paper 1	Detects isolated events; no event/aftershock clustering.	Add windowed spatiotemporal clustering using Spark/Flink.
	MongoDB capped storage limits historical analysis.	Implement Bronze–Silver–Gold Lakehouse with Delta Lake
	Uses a generic magnitude threshold with no aftershock modeling.	Compute cluster metrics: frequency, spatial spread, and decay rate.
Github Solution 1	Requires physical sensors; not scalable for large-area monitoring.	Use the USGS real-time feed for wider and easier earthquake coverage.
	Focuses on earthquake detection, not aftershock clustering and historical analysis.	Add spatiotemporal clustering + Bronze–Silver–Gold lakehouse for aftershock analysis.
Github Solution 2	Existing solution streams and displays earthquake data but never checks if events are related.	Adds real-time clustering logic to detect aftershock sequences as they form, plus a lakehouse for historical analysis.

References:

Paper 1: https://isprs-annals.copernicus.org/articles/IV-4-W2/215/2017/
Paper 2:
Github Solution 1: https://github.com/GeoShake/geoshake
Github Solution 2: https://github.com/25thOliver/Real-Time-Earthquake-CDC

Tracker Link:

https://docs.google.com/spreadsheets/d/1t_s9ceWBCumw9T-Ds-D1xpl7wj1Lrhdt4x-Lu2PSHDg/edit?gid=5934887#gid=5934887